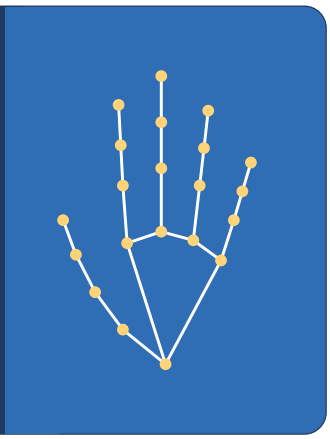
Gesture-Controlled UR5e Manipulator

Person-Aware Vision Pipeline, Control Flow and Safety Architecture

Author: Xamdamov Abu Bakr

System flowcharts · Version 1.1 · 3 November 2026 · validated on a physical UR5e
UR5e · Hikvision RTSP camera · MediaPipe · YuNet · URScript / RTDE



Abstract. This document specifies the architecture and the control logic of a vision-based human-robot interface in which an operator commands a Universal Robots UR5e arm with hand gestures. A fixed network camera is analysed on a laptop: the system finds and follows the operator, derives a work zone for his right hand from the size of his face, recognises the number of raised fingers, and, once a gesture has been held for three seconds, moves the robot to the corresponding taught pose. Robot motion is commanded through URScript and verified independently through read-only RTDE telemetry. All processes are presented as flowcharts that follow the swim-lane and staged-pipeline conventions of the reference literature on gesture-controlled robots. The complete system has been validated on a physical UR5e manipulator.

Contents

1. Introduction and Scope 2
2. System Overview 3
3. End-to-End Process (Swim-Lane Flowchart) 4
4. Perception Pipeline 5
5. Person Detection, Tracking and Active Zoom 5
6. Hand Gesture Recognition 7
7. Robot Motion Execution and Verification 8
8. Safety and Interlocks 10
9. Configuration Parameters 11
10. Comparison with the Baseline 11
11. Commissioning and Acceptance Tests 11
12. Conclusion and Limitations 12
Appendix A. Taught Waypoints on the Teach Pendant 13
Appendix B. Software Module Map 14
References 14

1. Introduction and Scope

Gesture interfaces remove the need for a teach pendant or keyboard in collaborative cells. The baseline for this work is the public *robodk_vision* example (Rebots-Spain), which maps one, two or three raised fingers to three RoboDK targets. That example is a proof of concept: it uses a blocking motion call inside the video loop, has no temporal filtering, no notion of *who* is gesturing, and no safety supervision. The present system keeps the intuitive finger-counting idea and the finger-state rule, but replaces everything around it.

Design objectives

- **Operate the real UR5e** (no simulation, no dry run) through URScript on port 30002, with all state read from RTDE on port 30004 (read-only).
- **Be person-aware:** detect one operator, zoom in and follow him, and accept gestures only from *his right hand* inside a zone defined relative to his face.
- **Be robust:** a gesture counts only after it has been held for 3 s (majority-voted over a sliding window), so flicker and passing hand motions cannot start a move.
- **Be verifiable:** arrival at a pose is confirmed from robot telemetry, never assumed; faults abort and disable gesture control.
- **Be modular:** perception, decision, robot interface and UI are separate Python modules (Appendix B).

Concept in one picture

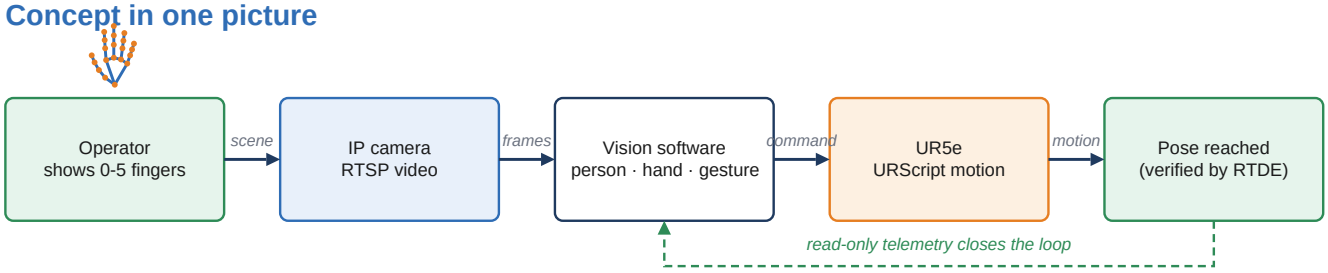


Figure 1. *Concept: a finger count shown by the operator becomes a verified robot motion.*

Table 1. *Scope of the system.*

In scope	Out of scope
Operator detection, zoom and follow; right-hand work zone; finger counting 0 - 5; 3 s confirmation; pose motion with RTDE verification	Collision-free path planning; grasping and tool control; multi-operator arbitration; replacing the robot's safety system

2. System Overview

The system has three tiers (Figure 1). The **sensing** tier delivers frames from an IP camera over RTSP; the **perception and decision** tier runs on a laptop with an NVIDIA GPU (NVDEC hardware decoding); the **actuation** tier is the UR5e controller. Commands flow to the robot on a single write channel (URScript); state flows back on a separate read-only channel (RTDE), which keeps monitoring independent from command issuing.

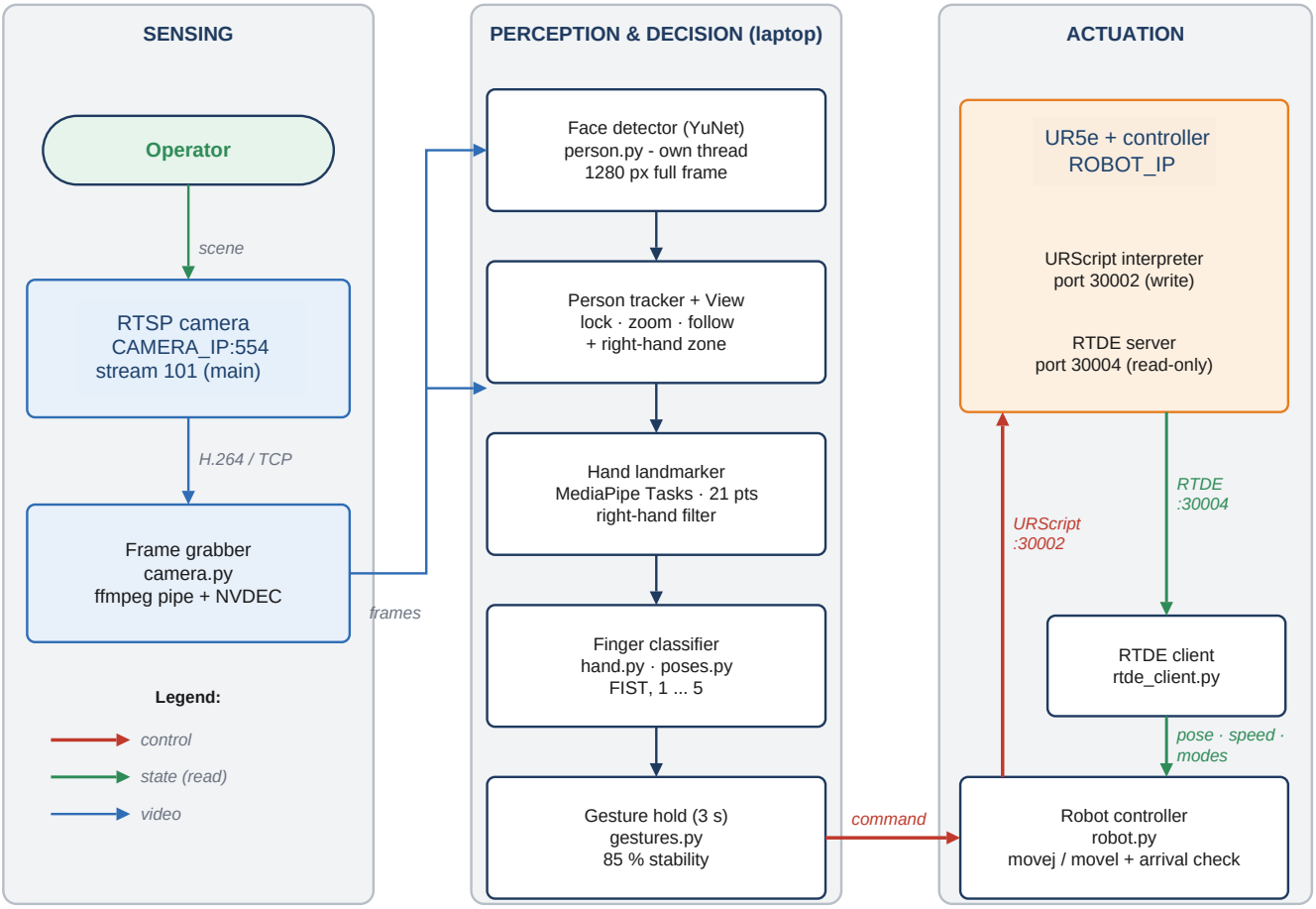


Figure 2. *Three-tier system architecture with the module responsible for each stage.*

Table 2. *Hardware and network configuration.*

Element	Setting	Purpose
IP camera	CAMERA_IP RTSP :554, stream 101 (main), TCP	Full-quality video; TCP avoids packet loss artefacts
Decoder	ffmpeg pipe with NVDEC (OpenCV fallback)	Lowest latency; GPU decoding
Robot	UR5e, ROBOT_IP	Actuator
Command channel	TCP :30002 (URScript), plain socket	Motion commands (`movej`, `movel`, `stopl`)
State channel	TCP :30004 (RTDE v2, 125 Hz), read-only	actual_TCP_pose, actual_TCP_speed, robot_mode, safety_mode

3. End-to-End Process (Swim-Lane Flowchart)

Figure 2 shows the complete control cycle across the perception lane, the decision-and-safety lane and the robot controller. Red arrows mark the command path from a recognised gesture to executed motion; the red retry bus on the left returns the process to frame acquisition whenever a condition is not met. In contrast to the reference flowchart, which ends after one command, the cycle is continuous and returns to *Idle* after every motion.

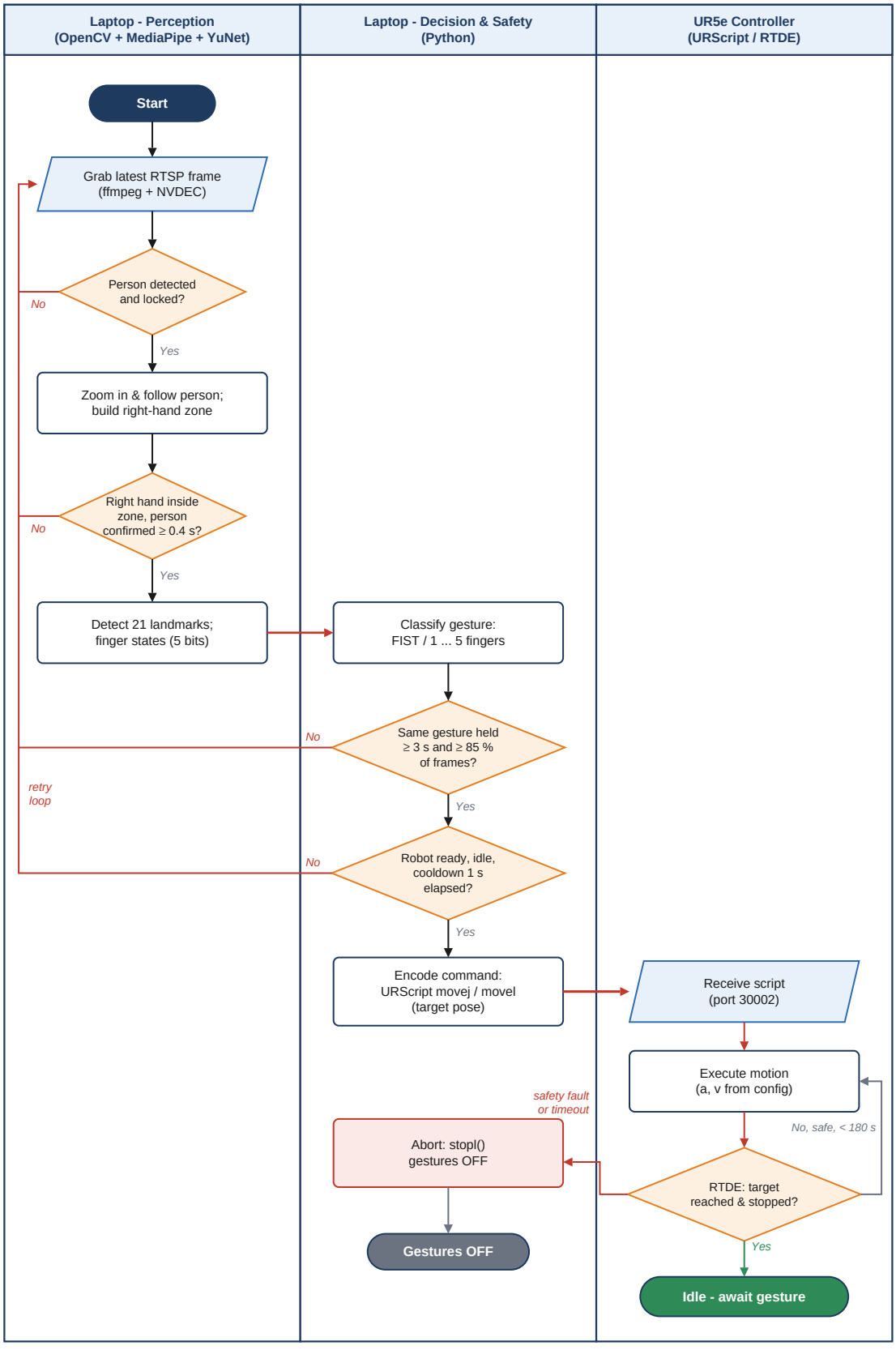


Figure 3. *Swim-lane flowchart of the gesture-to-motion process.*

4. Perception Pipeline

The perception chain (Figure 3) follows the staged organisation used for MediaPipe-based gesture applications: image acquisition, localisation (palm detector), key-point regression (hand landmark), gesture recognition and application response. Two stages are added in front of the hand model - person localisation and active zoom - because a fixed wide view makes hands too small for reliable landmarks at working distance.

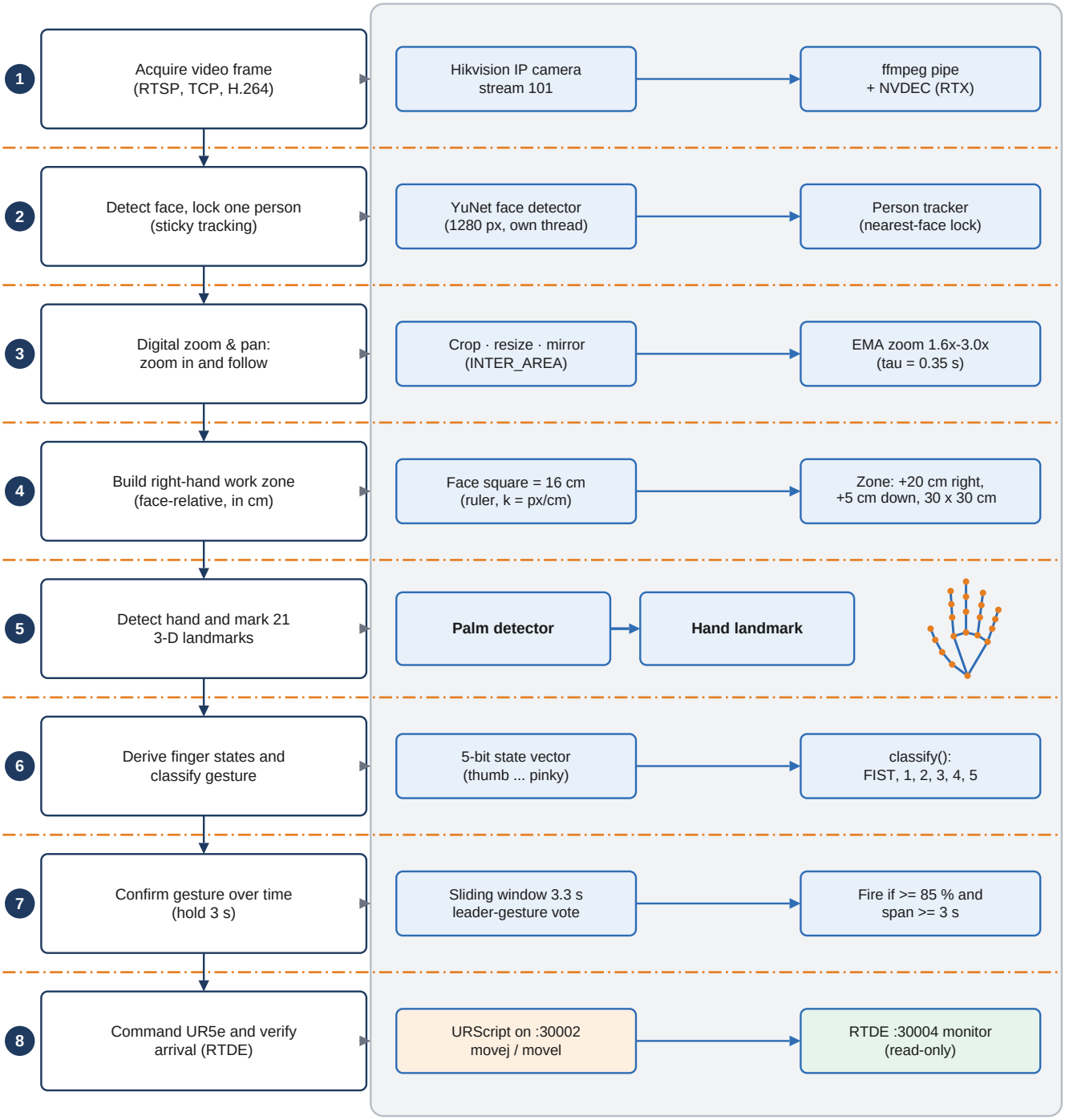


Figure 4. *Staged perception pipeline. Dash-dot lines separate the processing stages; the grey panel lists the modules realising each stage.*

5. Person Detection, Tracking and Active Zoom

Face detection runs in its own thread on the *whole* frame, downscaled to 1280 px width. Using the full frame (instead of the zoomed crop) lets the detector keep finding the operator while the view is moving, and detection never blocks the video display. The YuNet detector is used with a 0.5 score threshold; if it cannot be loaded, an OpenCV Haar cascade serves as fallback.

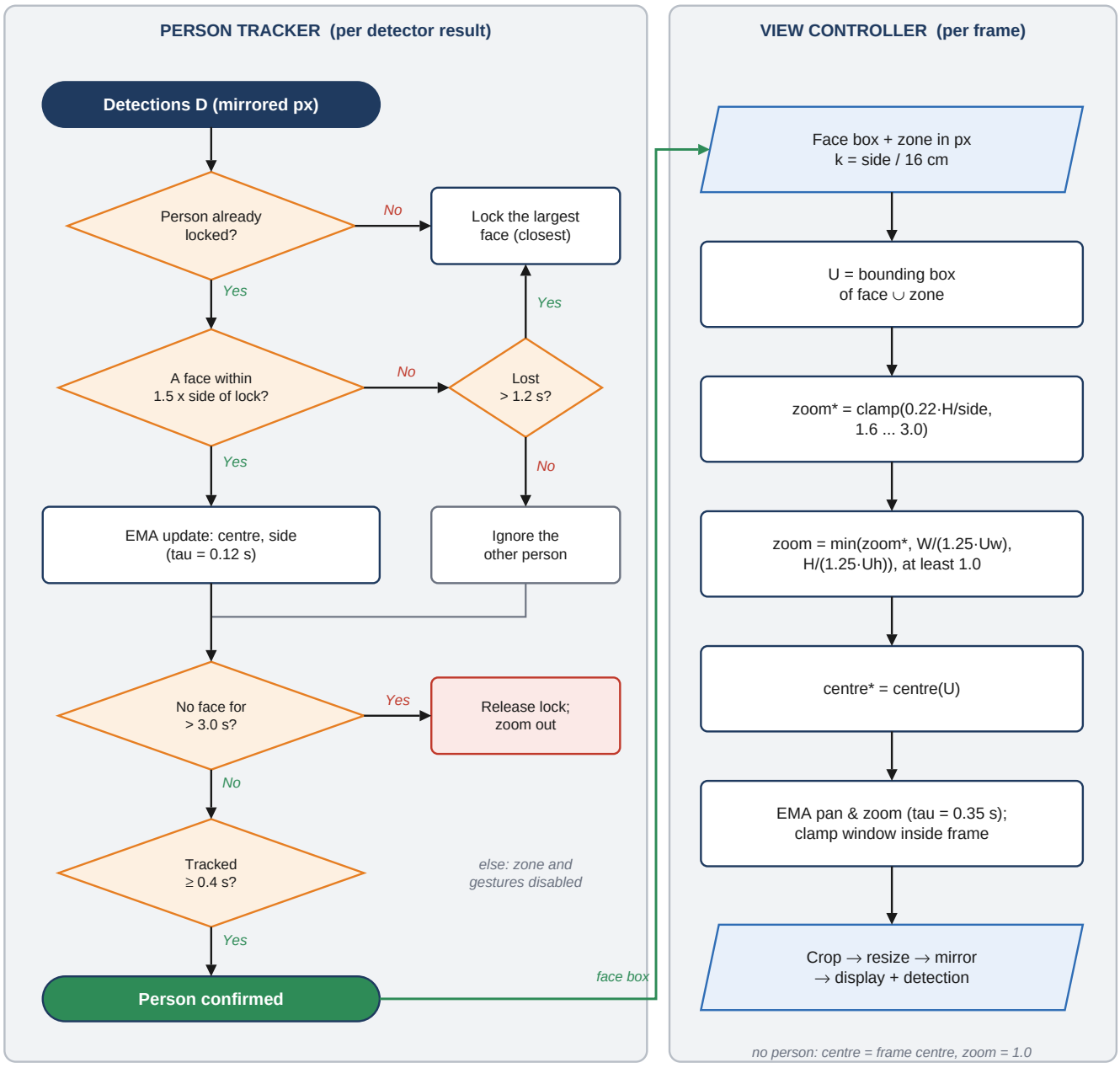


Figure 5. *Person tracker (left) and view controller (right).*

Tracking nuances

- **One operator.** The tracker locks the largest (closest) face and then follows the face nearest to the previous position (within 1.5 face sides). A second person walking by does not steal the lock; he is drawn as a thin grey box.
- **Blink tolerance.** If the detector misses a few frames the box is held for 1.2 s; after 3 s without a face the lock is released and the view zooms out.
- **Smooth camera.** Pan and zoom are exponentially smoothed (tau = 0.35 s) so the image glides rather than jumps. The window frames the face *and* the work zone together.
- **Constant on-screen face size.** The zoom is regulated so the face occupies about 22 % of the view height (limits 1.6x - 3.0x), which keeps the square at a constant on-screen size while the operator moves closer or further.

5.1 Work zone for the right hand

The zone is defined in centimetres relative to the face, using the face square as a ruler, assuming the standard anthropometric mean of 16 cm (actual faces vary). It starts 20 cm to the operator's right of the face square and 5 cm below its top edge, and is 30 x 30 cm. Because the display is mirrored, the operator's right is the right-hand side of the screen. Only a hand that (a) is classified as a right hand by MediaPipe and (b) has its palm centre inside the zone is analysed; all other hands are drawn in grey and ignored. The zone is disabled (orange) if it would leave the camera frame.

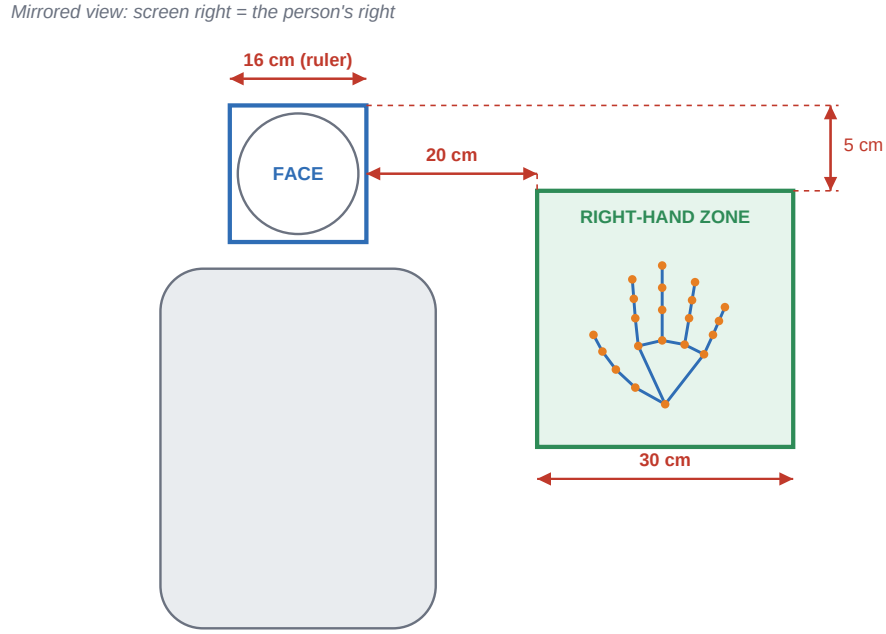
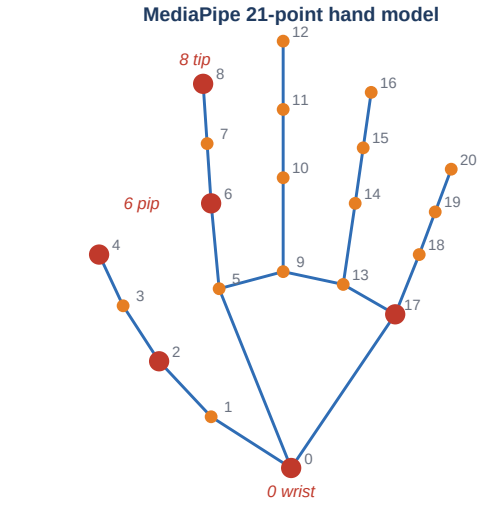


Figure 6. *Geometry of the right-hand work zone. All dimensions scale with the detected face size.*

6. Hand Gesture Recognition

Hand landmarks are obtained with MediaPipe's two-stage model (palm detector followed by a 21-point landmark regressor). The finger-state rule of the baseline script is retained because it is simple, calibration-free and rotation-tolerant (Figure 4).



Finger-state rule (adopted from the baseline script)

For tip $i \in \{8, 12, 16, 20\}$ (index, middle, ring, pinky):
up_i = 1 if $|tip_i - wrist(0)| > |pip_i - wrist(0)|$
with $pip_i = tip_i - 2$ (e.g. 8 vs 6)

Thumb (tip 4):
up = 1 if $|4 - pinky_MCP(17)| > |2 - pinky_MCP(17)|$

Distances are Euclidean in image pixels; the rule is rotation-tolerant and needs no calibration. The thumb is ignored for FIST and for 1-3 fingers; it separates 4 from 5.

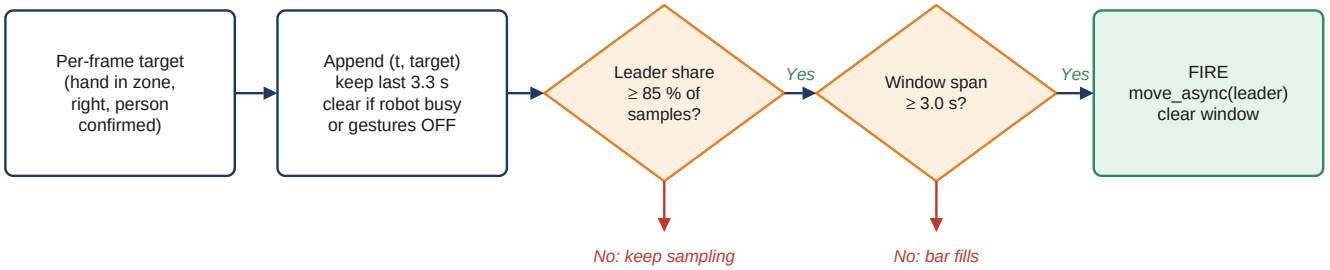
Figure 7. *Landmark numbering and the finger-up rule (distance of a finger tip versus its PIP joint from the wrist).*

Table 3. *Gesture-to-pose mapping (* = any value).*

Gesture	Finger state (thumb, index, middle, ring, pinky)	Target pose
Fist	(*, 0, 0, 0, 0) - thumb ignored	FIST - baseline pose P1
1 finger	(*, 1, 0, 0, 0)	Waypoint 1
2 fingers	(*, 1, 1, 0, 0)	Waypoint 2
3 fingers	(*, 1, 1, 1, 0)	Waypoint 3
4 fingers	(0, 1, 1, 1, 1)	Waypoint 4
5 fingers	(1, 1, 1, 1, 1)	Waypoint 5

6.1 Temporal confirmation

A pose is requested only when the same gesture has been held for three seconds. The decision uses a sliding window with majority voting (Figure 5).



Why a sliding window? A strict 'N identical frames' test resets on every one-frame flicker (typical for the thumb). Majority voting over 3.3 s tolerates up to 15 % outlier frames while the hand still has to be held ≥ 3 s, so waving through other gestures can never trigger a move. After a move, a 1 s cooldown re-arms the logic.

Figure 8. *Gesture confirmation logic. A progress bar on screen shows the remaining hold time.*

7. Robot Motion Execution and Verification

Motion is requested through a single-command URScript program. Because a program sent to port 30002 does not report completion, arrival is established from RTDE telemetry. Taught waypoints (recorded on the teach pendant, Appendix A) are executed with `movej` using their joint values, which avoids inverse-kinematics ambiguities and unexpected straight-line paths between distant poses (waypoint 5 lies close to the reach limit). The fist pose has no recorded joint values and is executed with `movel` using the requested Cartesian speed and acceleration; this straight-line motion has been validated in the physical workspace (Section 12).

Table 4. *Taught poses used by the controller.*

> Public edition: deployment-specific TCP and joint coordinates are intentionally omitted. Teach and verify each waypoint on your own robot using the safety procedures for that cell.

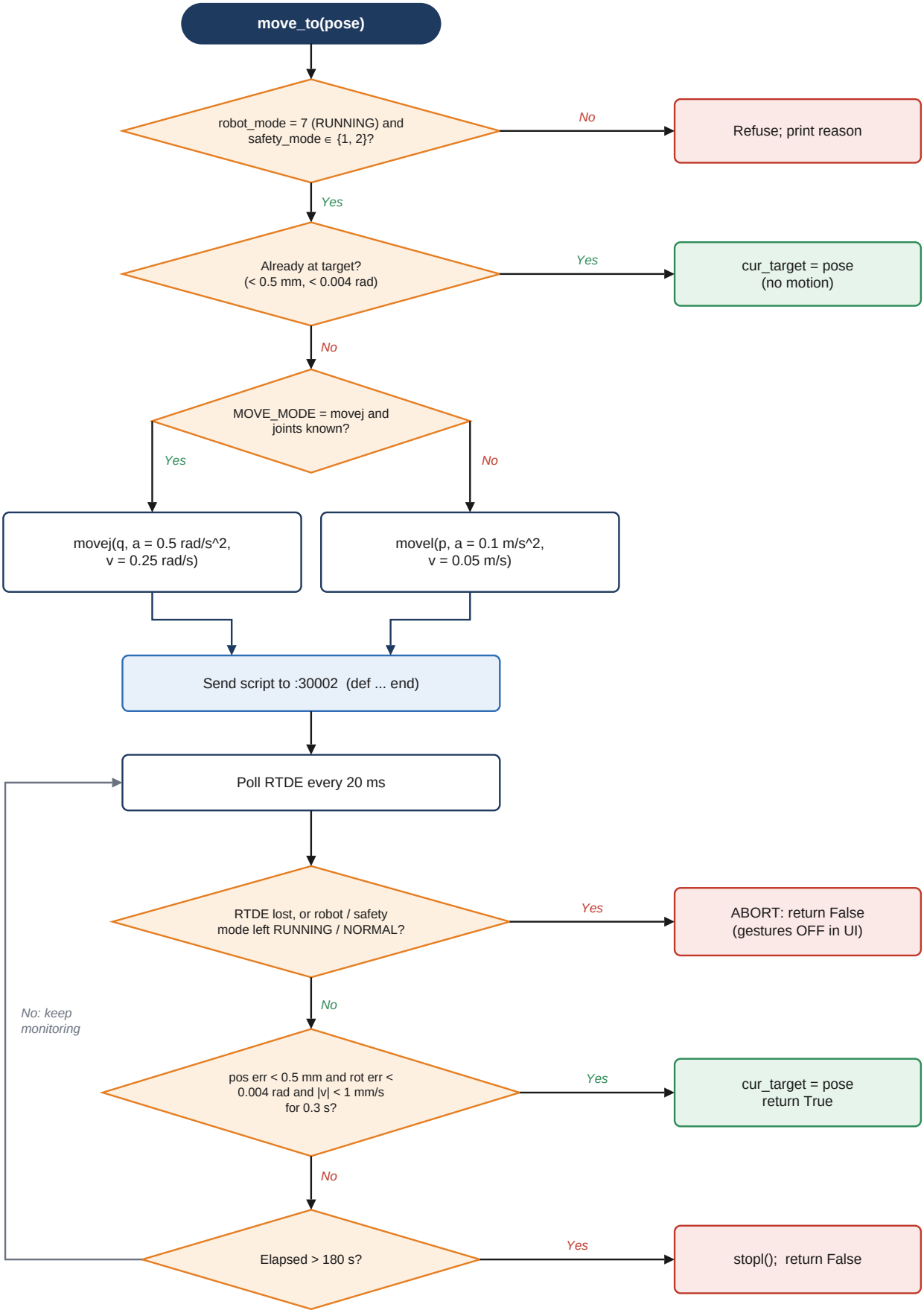


Figure 9. *Motion execution, safety supervision and arrival verification.*

8. Safety and Interlocks

The vision system is an *operator convenience*. Its software interlocks are not safety-rated functions: they **complement, but do not replace,** the certified hardware safety functions of the UR5e (emergency stop, protective stop, safety limits and safeguarding) required by **ISO 10218-1/-2** and, for collaborative operation, **ISO/TS 15066**. Risk assessment and safety configuration of the cell remain the integrator's responsibility. The interlocks below reduce the likelihood of unintended motion and act in addition to those hardware functions.

Table 5. *Safety interlocks.*

Interlock	Condition	Reaction
Operator presence	Face tracked ≥ 0.4 s; lock lost after 3 s	Gestures ignored; view zooms out
Hand authority	Right hand and palm centre inside the zone; zone inside frame	Other hands drawn grey and ignored
Intentionality	Same gesture ≥ 3 s, ≥ 85 % of frames	No motion until satisfied
Single motion	No new command while moving; 1 s cooldown	Window cleared during motion
Robot state	robot_mode = 7 and safety_mode ∈ {1, 2} before and during motion	Refuse / abort
Telemetry	RTDE alive	Gestures disabled on loss
Timeout	No arrival within 180 s	`stopl`, return failure
Manual override	Key **x** sends stopl and disables gestures; Esc / q quits; start requires typing YES	Immediate
Hardware (certified)	UR5e safety functions: emergency stop, protective stop, safety limits (ISO 10218-1/-2, ISO/TS 15066)	Always active, independent of this software

9. Configuration Parameters

Table 6. *Principal parameters (all defined in* `config.py`*).*

Parameter	Value	Meaning
MOVE_SPEED_M_S / MOVE_ACCEL_M_S2	0.05 / 0.1	Cartesian speed (m/s) and acceleration (m/s²) for movel
JOINT_SPEED_RAD_S / JOINT_ACCEL_RAD_S2	0.25 / 0.5	Joint speed and acceleration for movej
GESTURE_HOLD_S / COOLDOWN_S	3.0 / 1.0 s	Required hold time; pause after a move
POS_TOL_M / ROT_TOL_RAD / STOP_SPEED_M_S	0.0005 / 0.004 / 0.001	Arrival tolerances (position, orientation, speed)
MOVE_TIMEOUT_S	180 s	Maximum duration of one motion
FACE_DET_WIDTH / FACE_SCORE	1280 px / 0.5	Face search resolution and confidence
FACE_REAL_CM	16 cm	Ruler used to convert centimetres to pixels
ZONE_GAP_X_CM / ZONE_DROP_Y_CM / ZONE_SIZE_CM	20 / 5 / 30 cm	Position and size of the right-hand zone
FACE_TARGET_FRAC / MIN_TRACK_ZOOM / MAX_ZOOM	0.22 / 1.6 / 3.0	Auto-zoom regulation
VIEW_TAU_S / FACE_TAU_S	0.35 / 0.12 s	Smoothing time constants
FACE_HOLD_S / PERSON_CONFIRM_S / PERSON_RELEASE_S	1.2 / 0.4 / 3.0 s	Detector blink tolerance, lock confirmation, lock release

10. Comparison with the Baseline

Table 7. *Differences from the reference implementation.*

Aspect	Baseline (robodk_vision)	This system
Robot interface	RoboDK API (`MoveJ` to a target)	URScript :30002 + RTDE :30004 (UR5e)
Gesture set	1, 2, 3 fingers → 3 targets	Fist and 1 ... 5 fingers → 6 poses
Trigger	Immediate on detection	3 s hold, 85 % majority, cooldown
Execution	Blocking call inside the video loop	Worker thread; video never freezes
Completion	Returned by the simulator call	Verified from RTDE telemetry
Operator awareness	None - any hand in view	Face lock, zoom/follow, face-relative right-hand zone
Safety	None	Interlocks of Section 8
MediaPipe API	Legacy *solutions* API	Tasks API (HandLandmarker), model downloaded on first run

11. Commissioning and Acceptance Tests

The acceptance tests below were carried out on the physical UR5e with the emergency stop within reach. All tests were passed; no faults or glitches were observed.

Table 8. *Commissioning and acceptance tests (physical UR5e).*

#	Test	Expected result	Pass
1	Start program, check console	Camera stream OK; RTDE ready = True	Passed
2	Keys 0 - 5 (manual pose)	Robot reaches each pose; no protective stop	Passed
3	Walk into view	Face box, zoom-in and follow within ~1 s	Passed
4	Second person passes	Lock stays on the operator	Passed
5	Right hand in zone, show 3 fingers for 3 s	Progress bar fills, robot moves to waypoint 3	Passed
6	Left hand / hand outside zone	Ignored	Passed
7	Gesture held < 3 s, then changed	No motion	Passed
8	Press x during motion	Immediate stop, gestures OFF	Passed
9	Disconnect network cable	RTDE LOST; gestures disabled	Passed

12. Conclusion and Limitations

The system turns a finger count shown by a tracked operator into verified UR5e motion, with perception, decision and actuation separated into modules and supervised by independent telemetry. The complete system has been validated on a physical UR5e: all acceptance tests of Section 11 were passed and no glitches or faults were observed. Known limitations: gesture recognition degrades under strong back-light or when the hand is far from the camera; the centimetre scale relies on a standard anthropometric mean face size (16 cm), so absolute offsets vary by a few centimetres between people. The straight-line (`movel`) motion to the fist pose has been fully validated in the physical workspace and executes without collisions. Waypoints were transcribed from the teach pendant and should be re-checked after any re-teaching. Future work includes obstacle-aware path planning, speech confirmation and multi-operator hand-over.

Appendix A. Taught Waypoints on the Teach Pendant

Public edition: teach-pendant photographs are withheld because they contain installation-specific robot joint and TCP data. The original appendix is retained only in the private documentation.

Appendix B. Software Module Map

Table 9. *Module responsibilities.*

File	Responsibility
main.py	Main loop: frame → person → zone → hand → gesture → robot; on-screen UI and keys
config.py	All parameters (camera, robot, tolerances, person tracking)
poses.py	Taught poses, motion mode, finger-count classification
camera.py	RTSP capture with ffmpeg pipe/NVDEC and OpenCV fallback; latest-frame thread
person.py	YuNet face detector (thread), sticky Person tracker, smooth View (zoom/pan)
hand.py	MediaPipe HandLandmarker wrapper, finger-state rule, landmark helpers
gestures.py	Sliding-window hold logic (3 s, 85 %)
robot.py	URScript sender, arrival and safety supervision
rtde_client.py	Minimal read-only RTDE v2 client
models.py / ui.py	Model downloads; drawing helpers

References

[1] Rebots-Spain, *robodk_vision* - hand and face detection example with RoboDK (GitHub repository).

[2] F. Zhang et al., “MediaPipe Hands: On-device Real-time Hand Tracking,” arXiv:2006.10214, 2020.

[3] W. Wu et al., “YuNet: A Tiny Millisecond-level Face Detector,” *Machine Intelligence Research*, 2023.

[4] Universal Robots, *Real-Time Data Exchange (RTDE) Guide* and *The URScript Programming Language*.

[5] ISO 10218-1/-2 and ISO/TS 15066 - safety requirements for industrial and collaborative robots.